Matura-Test Informatik — 5. Jahrgang

Dauer: 5 Stunden (240 min Praxis + 60 min Theorie) **Hilfsmittel:** Visual Studio (offline), Microsoft-Doku (offline), eigene Code-Sammlung **Erlaubt:** Internet **nein**, KI-Tools **nein**, eigene Notizen **ja Punkte gesamt:** 100 (80 Praxis / 20 Theorie)

Übersicht

Teil	Inhalt	Zeit	Punkte
Aufgabe 1	TableScript — DSL für berechnete Tabellen	120 min	40
Aufgabe 2	TourPlaner — Client-Server mit Algorithmen	120 min	40
Theorie A	UML & Architektur	~25 min	7
Theorie B	Algorithmen & Komplexität	~20 min	7
Theorie C	Parser & Grammatik	~10 min	4
Theorie D	Netzwerk & Datenformate	~5 min	2

Bestehensgrenze: 50 Punkte. Die Aufgaben sind unabhängig voneinander und können in beliebiger Reihenfolge bearbeitet werden.

Aufgabe 1 — TableScript (40 Punkte)

Szenario

Du sollst einen kleinen Interpreter für eine selbst definierte Sprache namens **TableScript** schreiben. TableScript erlaubt es Benutzer:innen, eine Tabelle mit Spalten und Zeilen zu definieren, Spalten anzulegen, deren Inhalt aus einer Formel berechnet wird, und einfache Aggregate auf den Spaltenwerten auszuführen. Das Ergebnis wird in einer WPF-Oberfläche als Tabelle angezeigt.

Beispielprogramm (Eingabe)

```
TABLE "Bestellung"

COLUMN Produkt   TEXT
COLUMN Menge     ZAHL
COLUMN Preis     ZAHL
COLUMN Gesamt    FORMEL Menge * Preis
COLUMN MitMwSt   FORMEL Gesamt * (1 + 0,2)

ROW "Brot",  3, 2,50
ROW "Milch", 2, 1,80
ROW "Kaese", 1, 6,00

SUMME Gesamt
MITTEL Menge
```

```
ZEIGE
```

Erwartetes Ergebnis

Wenn obiger Code im Code-Eingabefeld steht und der Run-Button gedrückt wird, erscheint im Ausgabe-Bereich:

```
Bestellung
+---------+-------+-------+--------+----------+
| Produkt | Menge | Preis | Gesamt | MitMwSt  |
+---------+-------+-------+--------+----------+
| Brot    |   3   | 2,50  |  7,50  |   9,00   |
| Milch   |   2   | 1,80  |  3,60  |   4,32   |
| Kaese   |   1   | 6,00  |  6,00  |   7,20   |
+---------+-------+-------+--------+----------+
SUMME(Gesamt) = 17,10
MITTEL(Menge) = 2,00
```

(Die ASCII-Tabelle ist nur eine Veranschaulichung — eine WPF-DataGrid-Darstellung ist gewünscht.)

Sprachelemente von TableScript

Element	Beschreibung
`TABLE "name"`	Setzt den Tabellennamen (genau einmal pro Programm).
`COLUMN <name> TEXT`	Definiert eine Textspalte.
`COLUMN <name> ZAHL`	Definiert eine Zahlenspalte.
`COLUMN <name> FORMEL <ausdruck>`	Definiert eine berechnete Spalte; der Ausdruck wird pro Zeile ausgewertet.
`ROW <wert>, <wert>, ...`	Eine Zeile mit Werten in der Reihenfolge der TEXT/ZAHL-Spalten (Formelspalten werden übersprungen).
`SUMME <spalte>`	Summe einer Zahlen- oder Formelspalte.
`MITTEL <spalte>`	Arithmetisches Mittel einer Zahlen- oder Formelspalte.
`ZEIGE`	Zeichnet die Tabelle samt Aggregaten in der UI.

Formel-Ausdrücke

- Vier Grundrechenarten: `+`, `-`, `*`, `/`
- Klammerung mit `(` und `)`
- Operator-Precedence: `*` und `/` binden stärker als `+` und `-`
- Zahlenliterale mit Komma als Dezimaltrennzeichen (`0,2` ist gültig, `0.2` nicht)
- Bezüge auf andere Spalten per Spaltenname (Groß-/Kleinschreibung beachtet)
- Ein Formel-Ausdruck kann sich auch auf eine andere Formel-Spalte beziehen (siehe `MitMwSt` → `Gesamt`)

Sonstige Regeln

- Strings stehen immer in doppelten Anführungszeichen.

- Mehrere Leerzeilen und beliebig viele Spaces/Tabs sind erlaubt.
- Schlüsselwörter (`TABLE` , `COLUMN` , ...) sind GROSS geschrieben.

Anforderungen an die Lösung

#	Anforderung	Punkte
1	Tokenisierung des Eingabetextes in eine geeignete Token-Liste	6
2	Parser, der die sieben Anweisungstypen (`TABLE`, `COLUMN`, `ROW`, `SUMME`, `MITTEL`, `ZEIGE`) korrekt unterscheidet	8
3	Auswertung von `FORMEL`-Ausdrücken inkl. korrekter Operator-Precedence und Klammerung	8
4	Bezüge auf andere Spalten (auch Formelspalten) funktionieren	4
5	Anzeige der Tabelle in einer WPF-DataGrid (oder gleichwertig) mit allen Spalten	6
6	`SUMME`/`MITTEL` werden korrekt berechnet und unterhalb der Tabelle angezeigt	4
7	Mindestens drei verschiedene Fehlertypen werden erkannt und in einer Fehlerliste positioniert angezeigt (z. B. unbekannter Spaltenname, fehlende `)`, Typmismatch in `ROW`); das Programm darf in keinem Fall abstürzen	4

Eingabedateien zum Testen

Es gibt **keine** vorgegebenen Inputdateien — du schreibst den Test-Eingabetext direkt in das Code-Eingabefeld deines Programms.

Abgabe

- Vollständige Visual-Studio-Solution im Ordner `Abgabe/Aufgabe1/<deinName>/`
- Zip-Datei der Solution (ohne `bin` / `obj`)
- Das Projekt **muss** kompilieren und ausführbar sein

Aufgabe 2 — TourPlaner (40 Punkte)

Szenario

Du erweiterst eine vorgegebene Reiseplanungs-Anwendung um drei algorithmische Operationen, die über ein TCP-Netzwerk angefragt werden können. Der **Server** kennt eine Menge von Sehenswürdigkeiten ("POIs") sowie die Verbindungen zwischen ihnen. Der **Client** stellt diese auf einem Canvas dar und fragt drei Operationen beim Server an.

Starter-Projekt

Im Angabeprojekt `Angabeprojekt_TourPlaner/` findest du eine vorbereitete Visual-Studio-Solution mit drei Projekten:

- `NetworkLib/` — fertige TCP-Kommunikation (`Transfer` , `MSG` , `IReceiver`)
- `Server/` — Server mit fertigem CSV-Loader; **Stubs** für die drei Algorithmen

- `Client/` — WPF-UI mit fertigem Canvas-Rendering der POIs; **Stubs** für die drei Aktionen

Die Algorithmen-Klassen (`DijkstraSolver` , `KMeansSolver` , `PermutationsTourSolver`) sowie die drei Operations-Handler im Server-Receiver werfen aktuell `NotImplementedException` — das ist deine Aufgabe.

Datenbasis (`data/pois.csv` , `data/verbindungen.csv`)

Beide Dateien werden vom Server beim Start geladen. Format mit Semikolon-Trennzeichen:

```
# pois.csv
Id;Name;Kategorie;X;Y
1;Stephansdom;Sehenswuerdigkeit;520;310
2;Hofburg;Sehenswuerdigkeit;480;330
...

# verbindungen.csv
VonId;NachId;Distanz
1;2;120
2;3;90
...
```

Die Verbindungen sind ungerichtet (eine Zeile gilt in beide Richtungen).

Drei Operationen, die der Client anfordert

Operation 1 — Kürzester Pfad (Dijkstra)

Anfrage: Client wählt zwei POIs (Start und Ziel) per ComboBox und drückt "Pfad anzeigen". **Server:** Berechnet mit dem Dijkstra-Algorithmus den kürzesten Pfad im POI-Graph. **Antwort:** Liste der POI-IDs auf dem Pfad in Reihenfolge + Gesamtdistanz. **Visualisierung:** Der Pfad wird im Canvas in **Rot** hervorgehoben, die Gesamtdistanz wird in einem Textfeld angezeigt.

Operation 2 — Clustern (K-Means)

Anfrage: Client gibt eine Zahl `k` (z. B. 3) ein und drückt "Clustern". **Server:** Gruppiert alle POIs anhand ihrer X/Y-Koordinaten in `k` Cluster mit dem K-Means-Algorithmus. **Antwort:** Pro POI die zugewiesene Cluster-Nummer (0 bis k-1). **Visualisierung:** Die POIs werden im Canvas in **k unterschiedlichen Farben** dargestellt (je Cluster eine Farbe).

Operation 3 — Beste Reihenfolge (Permutationsgenerator)

Anfrage: Client wählt mit Mehrfach-Auswahl einige POIs (max. 8) aus und drückt "Beste Tour". **Server:** Erzeugt mit einem **selbst geschriebenen** Permutationsgenerator alle Reihenfolgen, in denen die ausgewählten POIs besucht werden können (der erste übergebene POI bleibt als Startpunkt fix), und sucht die mit der **kleinsten Gesamtdistanz** entlang der vorhandenen Verbindungen. Es werden also Pfad-Distanzen aus dem Graph aufsummiert (nicht Luftlinie). Wenn zwischen zwei POIs in der gewählten Reihenfolge kein Pfad existiert, ist diese Permutation ungültig. **Antwort:** Beste gefundene Reihenfolge der POI-IDs + Gesamtdistanz. **Visualisierung:** Die ausgewählten POIs werden im Canvas mit **fortlaufender Nummerierung** (1, 2, 3, ...) beschriftet, die Tour-Linie wird in **Blau** gezeichnet.

Anforderungen an die Lösung

#	Anforderung	Punkte

1	Server lädt CSVs, lauscht auf TCP-Port, loggt Anfragen	4
2	Operation 1 — Dijkstra korrekt implementiert (PriorityQueue oder gleichwertig)	9
3	Operation 1 — Visualisierung des Pfads im Canvas	3
4	Operation 2 — K-Means korrekt implementiert (Iterations-Logik mit Konvergenz/Iterationsobergrenze)	8
5	Operation 2 — Cluster-Färbung im Canvas	2
6	Operation 3 — Eigener Permutationsgenerator (rekursiv oder iterativ, **nicht** über LINQ-Bibliothek)	6
7	Operation 3 — Tour-Visualisierung mit Nummerierung	3
8	Robustheit: Ungültige Anfragen (unbekannte ID, keine Verbindung, k=0) → Fehler-Antwort, kein Crash	3
9	Zwei Clients können parallel verbunden sein und unabhängig Anfragen stellen	2

Hinweise

- Du darfst die Datenformate der `MSG`-Klasse erweitern, aber das bestehende Schema bleibt: ein `MSG`-Typ pro Anfrage- und Antwort-Variante.
- Die Visualisierung muss bei jeder neuen Anfrage von vorherigen Markierungen "aufgeräumt" werden.
- Der Permutationsgenerator soll generisch eine Liste übergeben bekommen und alle N! Reihenfolgen erzeugen können — die Beschränkung auf den fixen Startpunkt erledigt der Aufrufer.

Abgabe

- Erweiterte Visual-Studio-Solution im Ordner `Abgabe/Aufgabe2/<deinName>/`
- Zip-Datei der Solution (ohne `bin` / `obj`, **mit** `data/`-Ordner)
- Das Projekt **muss** kompilieren und ausführbar sein
- In der Solution starten: zuerst Server, dann Client (gerne auch zwei Clients)

Theorieteil (20 Punkte, 60 Minuten)

> *Bearbeite die Theorie auf einem **separaten Blatt**. Bezüge auf deine eigenen Aufgaben (1 oder 2) sind ausdrücklich erwünscht und werden positiv bewertet.*

Block A — UML & Architektur (7 Punkte)

Frage 1 — Klassendiagramm zu Aufgabe 1 (3 P)

Zeichne ein UML-Klassendiagramm der Parser-Hierarchie deines TableScripts. Es sollen mindestens enthalten sein:

- die abstrakte Basisklasse für Statements,
- mindestens vier konkrete Statement-Klassen,
- die Hierarchie der Formel-Auswertung (Precedence-Klassen, mind. zwei Ebenen),
- Beziehungen (Vererbung, Aggregation, Komposition) korrekt eingezeichnet,

- die wichtigsten Felder und Methoden mit Sichtbarkeit (`+` , `-` , `#`).

Frage 2 — Sequenzdiagramm zu Aufgabe 2 (3 P)

Zeichne ein UML-Sequenzdiagramm für eine **"Beste Tour"**-Anfrage. Beteiligte Komponenten:

- Benutzer:in / Client-UI,
- Client- `Transfer` ,
- Server- `Transfer` (bzw. Server-Receiver),
- `PermutationsTourSolver` .

Zeige die zeitliche Abfolge inklusive Antwort-Pfad zurück zur UI.

Frage 3 — Architekturvergleich (1 P)

Erkläre in 4–6 Sätzen, warum für Aufgabe 2 eine Client-Server-Architektur sinnvoll ist, obwohl die ganze Anwendung auch in einem einzigen Programm laufen könnte. Nenne mindestens **zwei Vorteile** und **einen Nachteil**.

Block B — Algorithmen & Komplexität (7 Punkte)

Frage 4 — Dijkstra-Trockenlauf (2 P)

Gegeben ist folgender Graph:

```
       (4)
  A ———————— B
  |          /|
(2)|      (1) |(5)
  |     /    |
  D ———————— C
       (3)
  |
(7)|
  |
  E ————————  (Verbindung E-C mit Gewicht 6)
       (6)
```

Kantenliste: A–B(4), A–D(2), B–C(5), B–D(1), D–C(3), D–E(7), C–E(6)

Führe Dijkstra vom Knoten **A** aus per Hand durch und trage in eine Tabelle pro Iteration ein:

- welcher Knoten als nächstes finalisiert wird,
- die aktuellen Distanzen zu allen Knoten,
- den Vorgänger-Knoten jedes Knotens.

Frage 5 — K-Means in Worten (2 P)

Beschreibe in eigenen Worten (max. 10 Zeilen) den Ablauf des K-Means-Algorithmus, inklusive:

- Initialisierung,
- Iterationsschritte,
- Abbruchbedingung,
- **eine** Schwäche des Algorithmus.

Frage 6 — Permutationen — Komplexität (1 P)

Wie viele Permutationen entstehen für **N = 6** POIs? Wie viele für **N = 10**? Begründe in einem Satz, warum die Aufgabe `N ≤ 8` auf 8 begrenzt ist.

Frage 7 — Pseudocode (2 P)

Schreibe Pseudocode für einen **rekursiven** Permutationsgenerator, der eine Liste von Elementen übergeben bekommt und alle möglichen Anordnungen erzeugt. Der Pseudocode soll keine Klassen-spezifischen C#-Konstrukte enthalten (`Permutiere(rest, prefix)` oder ähnlich).

Block C — Parser & Grammatik (4 Punkte)

Frage 8 — EBNF-Grammatik (2 P)

Schreibe in EBNF die Grammatik für die `COLUMN` -Anweisung deines TableScripts (inklusive der Variante mit `FORMEL`). Verwende die Symbole `=` , `|` , `[ ]` , `{ }` , `( )` korrekt. Du darfst nicht-terminale Symbole wie `<Bezeichner>` , `<Zahl>` , `<Ausdruck>` voraussetzen — du musst sie nicht weiter definieren.

Frage 9 — Tokenisierung (1 P)

Welche Tokens entstehen aus folgender TableScript-Zeile? Liste **pro Token** Typ und Wert auf:

```
COLUMN Gesamt FORMEL (Menge + 1) * Preis
```

Frage 10 — Operator-Precedence (1 P)

Warum reicht es nicht, die Tokens einer Formel einfach von links nach rechts auszuwerten? Begründe an einem **konkreten Beispielausdruck** aus TableScript und gib das Ergebnis bei naiver Auswertung sowie das mathematisch korrekte Ergebnis an.

Block D — Netzwerk & Datenformate (2 Punkte)

Frage 11 — TCP vs. UDP (1 P)

Erkläre in maximal 4 Sätzen den zentralen Unterschied zwischen TCP und UDP und begründe, warum für die TourPlaner-Anwendung TCP gewählt wurde.

Frage 12 — Nachrichtenformat & Serialisierung (1 P)

Skizziere die Felder einer `MSG` -Klasse, die für **alle drei** Operationen (`KÜRZESTER_PFAD` , `CLUSTERN` , `BESTE_REIHENFOLGE`) sowie deren Antworten ausreicht. Begründe stichwortartig, warum du jedes Feld brauchst. Nenne anschließend ein **Serialisierungsformat** (z. B. XML oder JSON) mit je einem **Vor- und einem Nachteil**.

Allgemeine Hinweise

- Schreib deinen **Namen** auf jedes abgegebene Blatt und in jede Solution (in einer `README.txt`).
- Wenn du eine Aufgabe nicht vollständig schaffst, **dokumentiere kurz, was funktioniert und was nicht** — das gibt Teilpunkte.
- Lieber eine kompakte, robuste Lösung als eine ambitionierte, die nicht kompiliert.
- Viel Erfolg!